

Code Explanation

Batch Processing Test Suite Explanation

This test suite is designed to validate the functionality of batch processing related functions, including data cleaning, order data processing, and updating SCD Type 2 tables. The test suite utilizes PySpark for data processing and unittest for testing.

Overview

The provided code is a test suite written in Python using the unittest framework. It tests various batch processing functions, including data cleaning, order data processing, and updating SCD Type 2 tables. The tests are performed using PySpark DataFrames.

Step 1: Setting Up the Test Environment

The test environment is set up by creating a SparkSession and defining the schemas for customer and order data. This step is crucial for creating DataFrames that will be used throughout the tests.

```
@classmethod
def setUpClass(cls):
    cls.spark = SparkSession.builder
        .appName("TestBatchProcessing")
        .master("local[*]")
        .getOrCreate()

    cls.customer_schema = StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])

    cls.order_schema = StructType([
        StructField("OrderId", StringType(), True),
        StructField("ItemName", StringType(), True),
        StructField("PricePerUnit", DoubleType(), True),
        StructField("Qty", IntegerType(), True),
        StructField("Date", DateType(), True),
        StructField("CustId", StringType(), True)
    ])
]
```

Step 2: Creating Sample Data and DataFrames

Sample customer and order data are created, and DataFrames are generated using the defined schemas. This step is essential for testing the batch processing functions.

```
customer_data = [
    ("C001", "John Doe", "john@example.com", "North"),
    ("C002", "Jane Smith", "jane@example.com", "South"),
    ("C003", "Bob Johnson", "bob@example.com", "East"),
    ("C004", None, "alice@example.com", "West"),
    ("C001", "John Doe", "john@example.com", "North") # Duplicate
]

order_data = [
```

```

        ("0001", "Laptop", 1000.0, 2, datetime.date(2023, 1, 15), "C001"
    ),
        ("0002", "Phone", 500.0, 1, datetime.date(2023, 1, 20), "C002"),
        ("0003", "Tablet", 300.0, 3, datetime.date(2023, 1, 25), "C003")
    ],
        ("0004", "Headphones", 100.0, None, datetime.date(2023, 1, 30),
"C001"),
        ("0002", "Phone", 500.0, 1, datetime.date(2023, 1, 20), "C002")
    ]
    # Duplicate
]

cls.customer_df = cls.spark.createDataFrame(customer_data, cls.customer_schema)
cls.order_df = cls.spark.createDataFrame(order_data, cls.order_schema)

```

Step 3: Testing Data Cleaning Functionality

The `test\_clean\_data` method tests the `clean\_data` function by verifying that it removes null values and duplicates from the customer and order DataFrames.

```

def test_clean_data(self):
    clean_customer_df = clean_data(self.customer_df)
    self.assertEqual(clean_customer_df.filter(col("Name").isNull()).count(), 0)
    self.assertEqual(clean_customer_df.count(), 3)

    clean_order_df = clean_data(self.order_df)
    self.assertEqual(clean_order_df.filter(col("Qty").isNull()).count(), 0)
    self.assertEqual(clean_order_df.count(), 3)

```

Step 4: Testing Order Data Processing

The `test\_process\_order\_data` method tests the `process\_order\_data` function by checking if it correctly adds a `TotalAmount` column to the order DataFrame and handles null values.

```

def test_process_order_data(self):
    processed_order_df = process_order_data(self.order_df)
    self.assertTrue("TotalAmount" in processed_order_df.columns)
    row = processed_order_df.filter(col("OrderId") == "0001").first()
    self.assertEqual(row["TotalAmount"], 2000.0)
    row_null = processed_order_df.filter(col("OrderId") == "0004").first()
    self.assertIsNone(row_null["TotalAmount"])

```

Step 5: Testing SCD Type 2 Table Update

The `test\_update\_scd\_type2\_table` method tests the `update\_scd\_type2\_table` function. Although this test is currently incomplete due to the reliance on Spark SQL operations on Delta tables, it checks that the function does not raise any exceptions.

```
def test_update_scd_type2_table(self):
    try:
        # This would fail in a unit test environment but would work
        # in Databricks
        # update_scd_type2_table(self.spark, clean_customer_df, processed_order_df)
        pass
    except Exception as e:
        self.fail(f"update scd type2 table raised exception {e}")
```

Step 6: Tearing Down the Test Environment

The test environment is torn down by stopping the SparkSession.

```
@classmethod
def tearDownClass(cls):
    cls.spark.stop()
```